

REST API

Representational State Transfer - concepts, architecture & diagrams

A practical reference guide

1. What is a REST API?

A **REST API** (Representational State Transfer Application Programming Interface) is an architectural style that is widely treated as the standard for designing and building the networked applications that power the web. It provides a set of rules and constraints that, when followed, produce web services that are **simple**, **scalable** and **easy to integrate**.

REST is not a protocol or a library; it is a set of design principles. It was described by Roy Fielding in his 2000 doctoral dissertation, and it builds directly on the existing mechanics of HTTP: URLs, methods, headers and status codes.

2. REST API defined

A REST API is an API that conforms to the design principles of the REST architectural style. At its core, REST revolves around **resources**: any piece of information such as a user, a product, a document or a collection of items. Each resource is addressed by a URI and manipulated through a small, fixed set of HTTP methods.

Key idea: In REST you think in *nouns* (resources such as `/users`), not *verbs* (such as `/getUser`). The verb comes from the HTTP method.

3. How does a REST API work?

Every interaction follows the same four-step cycle shown below.

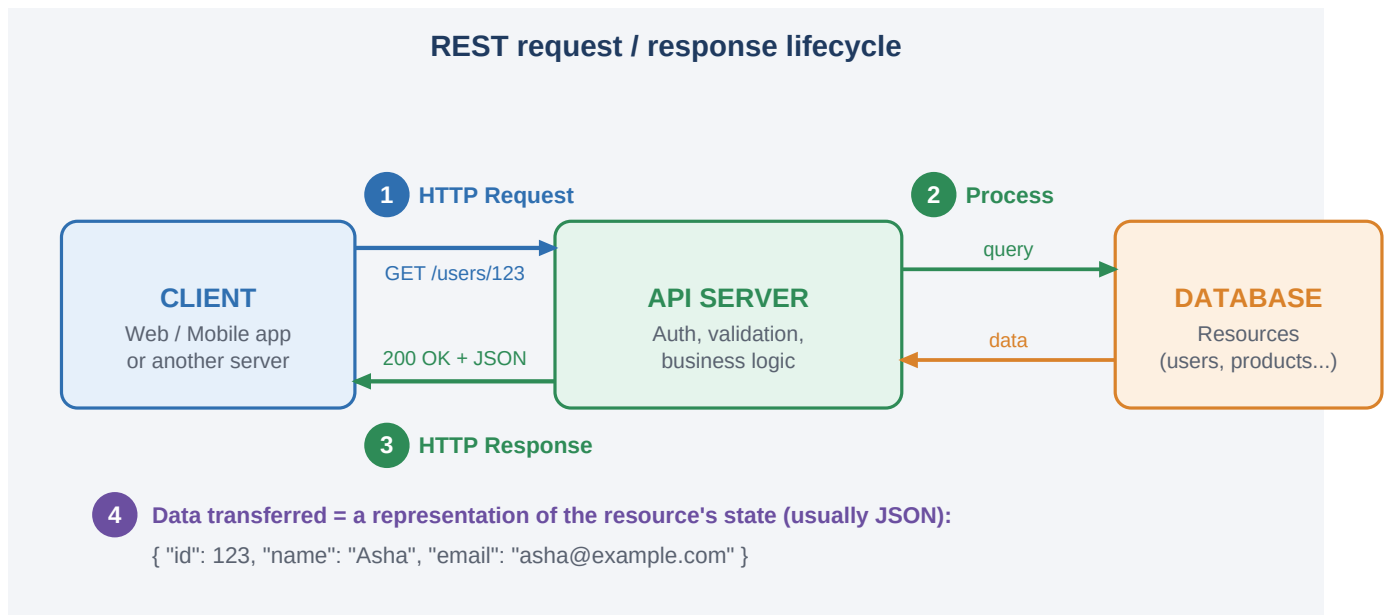


Figure 1 - The client sends a request, the server processes it (often against a database) and returns a response.

1. Client sends request

A client application (mobile app, web browser or another server) initiates a request to perform an action. The request goes to a specific URL and includes an **HTTP method**, **headers** carrying metadata (e.g. Authorization, Content-Type) and sometimes a **request body** with data.

2. Server processes request

The server receives and validates the request, confirming the client is **authenticated** (who are you?) and **authorized** (are you allowed to do this?). It then processes the request: retrieving data from a database, creating a new resource or updating an existing one.

3. Server sends response

The server returns a response containing an **HTTP status code** indicating the outcome (200 OK for success, 404 Not Found if the resource doesn't exist, 401 Unauthorized for authentication issues), along with any requested data in the response body.

4. Data transfer

What travels between client and server is a **representation** of the resource's state. **JSON** is the most common format because it is lightweight, human-readable and easy for programming languages to parse.

Example: a complete request and response

```
GET /v2/pet/findByStatus?status=available HTTP/1.1
Host: petstore.swagger.io
Accept: application/json
Authorization: Bearer eyJhbGciOi...

HTTP/1.1 200 OK
Content-Type: application/json

[
  { "id": 10, "name": "doggie", "status": "available",
    "category": { "id": 1, "name": "Dogs" } }
]
```

4. REST API architecture and components

The power and scalability of REST come from a set of architectural constraints that guide its design. A system that adheres to these constraints is called **RESTful**. The model below shows how the pieces fit together.

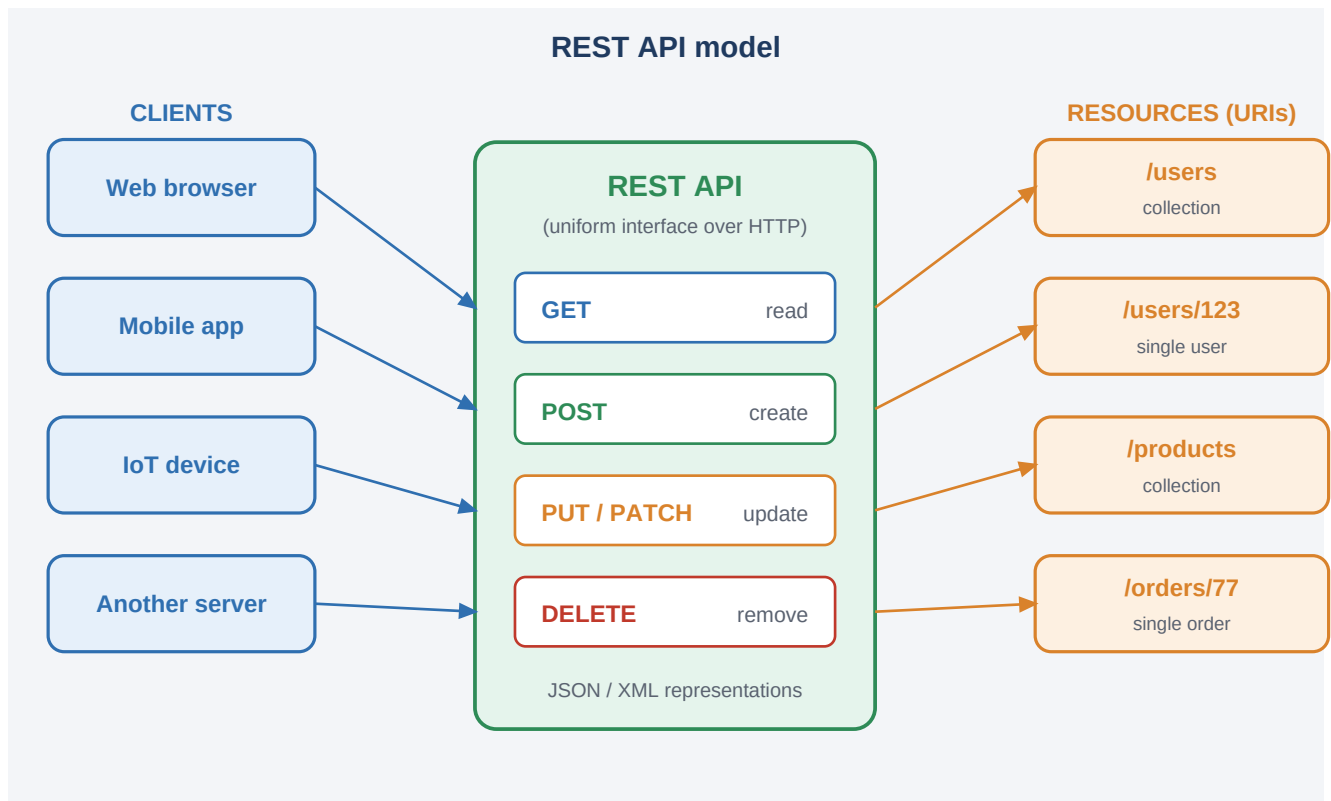


Figure 2 - REST API model: many kinds of clients use the same HTTP methods to act on resources identified by URIs.

4.1 Resources

A resource is the fundamental concept in REST. It is an object with a type, associated data, relationships to other resources and a set of methods that operate on it. In an e-commerce API, a *product* or a *customer* is a resource. Resources can be single items (/users/123) or collections (/users), and can be nested to express relationships (/users/123/orders).

4.2 URIs (Uniform Resource Identifiers)

Each resource is uniquely identified by a URI. A well-designed API uses clean, descriptive and consistent URIs. For example, `/users` identifies the collection of users and `/users/123` identifies the user with ID 123.

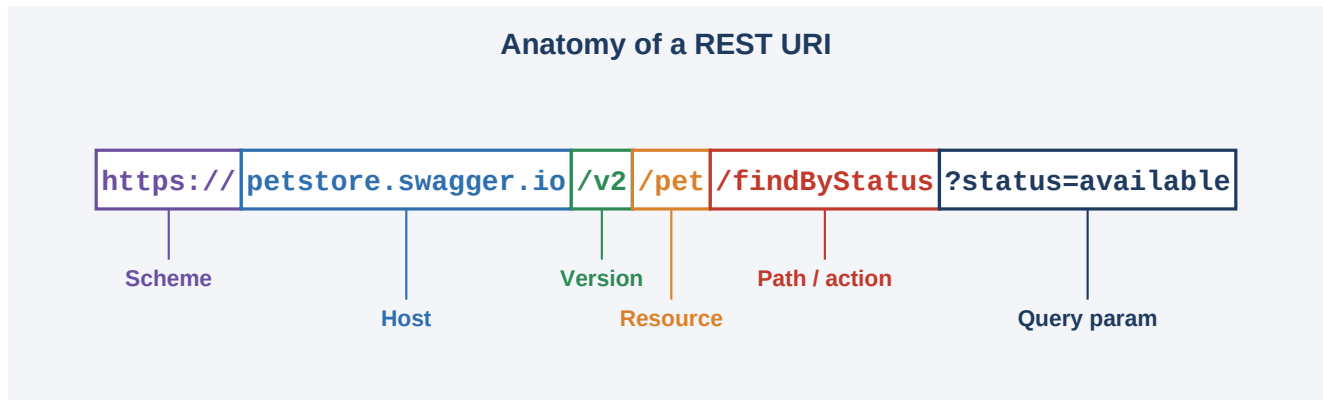


Figure 3 - Breaking down `https://petstore.swagger.io/v2/pet/findByStatus?status=available`

Good practice	Avoid	Why
<code>/users</code>	<code>/getAllUsers</code>	The method (GET) already expresses the action
<code>/users/123/orders</code>	<code>/orders?userFor=123&type=list</code>	Nesting shows the relationship clearly
Plural nouns, lowercase, hyphens	<code>/User_Profile</code>	Consistency makes APIs predictable
<code>/v1/users</code>	Breaking changes without a version	Versioning protects existing clients

4.3 Representations

A client never touches a resource directly; it works with a **representation** of it. The most common format is JSON, but XML, HTML or others can also be used. The client states its preference with the `Accept` header and the server labels the response with `Content-Type`. The same resource can therefore be served in several formats.

4.4 Statelessness

Every request must contain all the information the server needs to understand and process it. The server stores **no client context or session state** between requests; authentication typically travels with each request as a token. This is what makes REST APIs highly scalable: any server can handle any request.

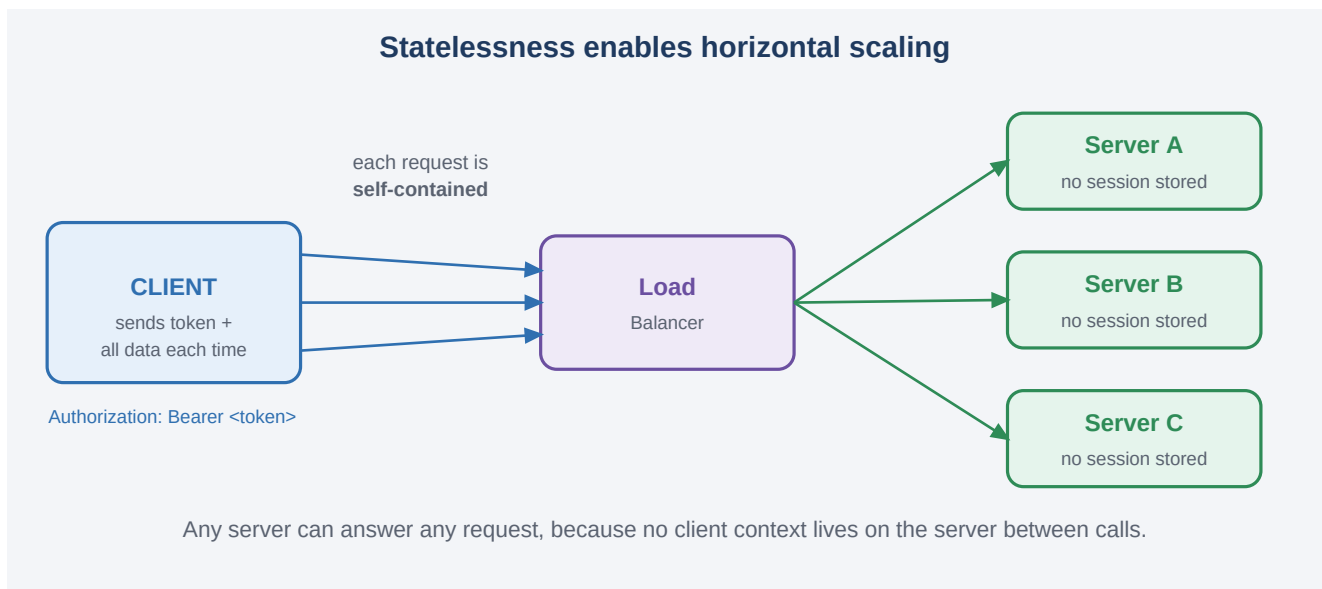


Figure 4 - Because requests are self-contained, a load balancer can route each one to any available server.

4.5 Client-server architecture

The client and server are separate, with the API acting as the contract between them. The client handles the user interface; the server handles data storage and business logic. As long as the API contract is kept, each side can evolve, be redeployed or be rewritten independently.

4.6 Uniform interface

REST requires a consistent, uniform interface between client and server, achieved through four elements:

Element	Meaning
Resource identification	Every resource is identified by a URI.
Manipulation through representations	Clients change resources by sending representations (e.g. a JSON body).
Self-descriptive messages	Each message carries enough metadata (method, headers, status) to be understood on its own.
HATEOAS	Hypermedia As The Engine Of Application State: responses include links telling the client what it can do next.

```

{
  "id": 123, "name": "Asha",
  "links": [
    { "rel": "self", "href": "/users/123" },
    { "rel": "orders", "href": "/users/123/orders" }
  ]
}
  
```

A HATEOAS-style response: links point the client to related actions.

4.7 The complete set of constraints

Besides the constraints above, the REST style also defines **cacheability** (responses declare whether they can be cached, reducing load and latency), a **layered system** (clients cannot tell whether they talk to the real server or an intermediary such as a gateway or load balancer) and the optional **code on demand**

(servers may send executable code, such as JavaScript, to clients).

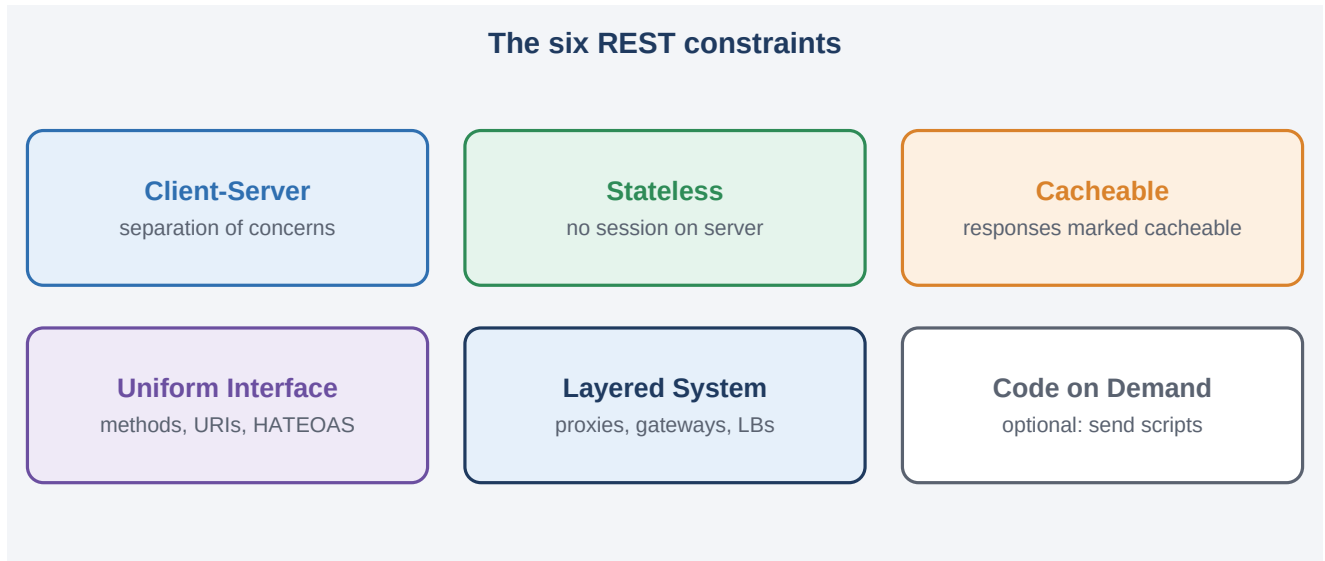


Figure 5 - An API that satisfies these constraints is called RESTful.

5. HTTP methods (CRUD mapping)

Method	CRUD	Example	Purpose	Idempotent?
GET	Read	GET /users/123	Retrieve a resource or collection	Yes
POST	Create	POST /users	Create a new resource	No
PUT	Update / Replace	PUT /users/123	Replace the entire resource	Yes
PATCH	Update (partial)	PATCH /users/123	Modify selected fields only	Not guaranteed
DELETE	Delete	DELETE /users/123	Remove the resource	Yes

Idempotent means making the same request several times has the same effect as making it once, which makes retries safe. GET is also **safe**: it should never change server data.

6. HTTP status codes

Range	Category	Common codes
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 304 Not Modified
4xx	Client error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity, 429 Too Many Requests
5xx	Server error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

401 vs 403: 401 means the client is not authenticated (missing or invalid credentials); 403 means the client is authenticated but not allowed to perform the action.

7. Why REST? Benefits at a glance

Benefit	Comes from
Scalability	Statelessness, caching and layered systems
Simplicity	Standard HTTP methods and status codes everyone already knows
Flexibility	Multiple representations (JSON, XML) and independent client/server evolution
Interoperability	Works with any language or platform that speaks HTTP
Testability	Each request is self-contained, so it can be tested in isolation with tools like Postman, curl or automated test suites

8. Quick summary

- REST is an architectural style, built on HTTP, centred on **resources**.
- Resources are identified by **URIs** and exchanged as **representations** (mostly JSON).
- HTTP **methods** express the action; **status codes** express the outcome.
- **Statelessness** and the **client-server** split make REST scalable and independently evolvable.
- A **uniform interface** (including HATEOAS) keeps every API predictable.